Verification of Concurrent Systems

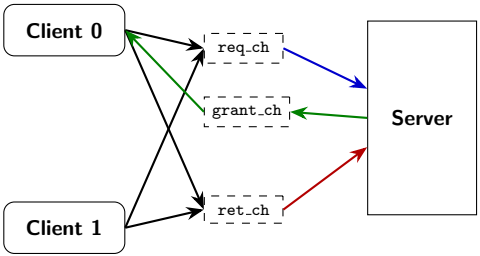
Sangeerthana K

ACM Isoft Ascent Intern, IISc Bangalore
Guide: Prof. Komondoor V. Raghavan

SERI 2026

Model Checking with SPIN

Client-Server Model



- **Single Request Bound:** Each client can issue at most a maximum of 1 active request at any given time.
- **Non-Deterministic Selection:** The central Server picks among pending client request messages non-deterministically to process and issue grants.
- **Release Cycle:** After processing inside the critical section, the corresponding client explicitly transmits a return flag back to the server.

Client-Server: Promela Model & LTL Specs

Simplified Promela Snippet

```
active proctype Client(byte id) {
  do
  :: req_ch!id; grant_ch?eval(id);
     in_cs[id] = true;
     /* Critical Section */
     in_cs[id] = false;
     ret_ch!id;
  od
}
active proctype Server() {
  do
  :: if
     :: req_ch?0 -> grant_ch!0; ret_ch?eval
        (0);
     :: req_ch?1 -> grant_ch!1; ret_ch?eval
        (1);
     fi
  od
}
```

LTL Specifications

- **Safety (Mutual Exclusion):**

```
ltl safe { []!(in_cs[0] && in_cs
    [1]) }
```

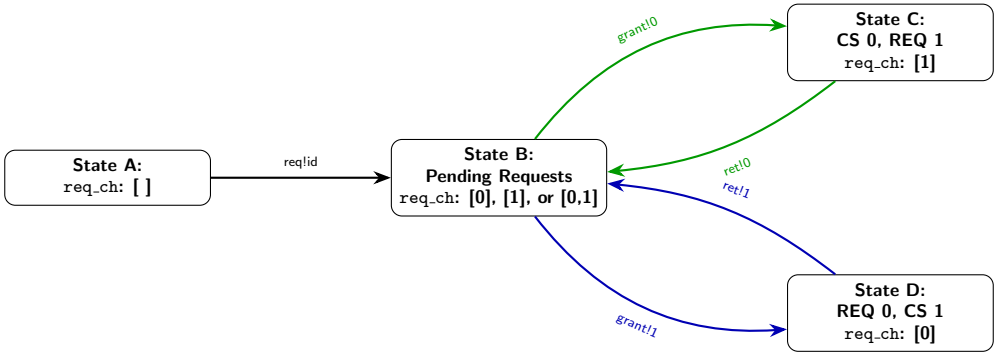
SPIN Result: **Passed**

- **Liveness (Request - Grant):**

```
ltl req_grant { [] (req_ch?[1] ->
     <> grant_ch?[1]) }
```

SPIN Result: **Failed**

Global State Automaton: Trace Analysis



Passing Progress Trace

- $A \to B \xrightarrow{\text{grant!0}} C \to B \xrightarrow{\text{grant!1}} D \ldots$

Failing Starvation Trace

- $A \to B \xrightarrow{\text{grant!0}} C \to B \xrightarrow{\text{grant!0}} C \ldots$

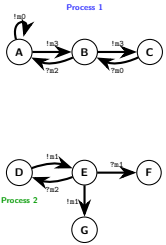
Transition to Abstraction: Karp-Miller

Bridging Explicit State Checking to Abstraction

- **The Limits of SPIN Explicit State Checking:**
  - SPIN tracks absolute buffer sizes and concrete trace histories (e.g., `req_ch: [0,1]`).
  - If we remove the fixed size request channel restriction, the state space explodes to an **infinite size**, completely breaking exhaustive verification engines.
- **Transition to Structural Abstraction:**
  - To verify general models, we need to map communication states into safe structural abstractions.
  - **Vector Addition Systems (VAS)** achieve this by transforming individual message events into numeric token counters.
- **Handling the Infinite State Space:**
  - The **Karp-Miller Algorithm** systematically tracks these unbounded counters by accelerating loops using abstract infinity (ω).

Studying the Classical Karp-Miller Algorithm

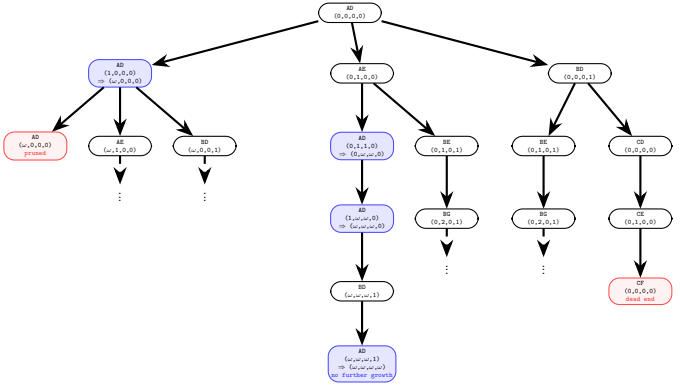
Process Automata



Core Tree Rules:

- **Acceleration Rule:** If transitioning to a new queue y from an ancestor x (both same product state) where $y \geq x$, the strictly increasing coordinate is accelerated to ω to guarantee termination.
- **Subsumption Rule:** If a newly generated queue state y is equal to or covered by an existing state x ($y \leq x$) (both same product state), the branch is pruned to prevent infinite exploration in karp miller.

Karp-Miller Tree



Advanced Framework & Current Progress

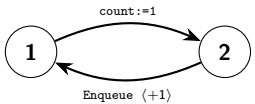
Advanced Research Framework Developed in the Lab

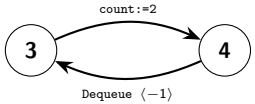
Prior to my internship work, the lab had developed a unified static analysis approach designed specifically to target concurrent behaviors globally:

- **Joint Verification Architecture:** The engine simultaneously takes both the **message queue configurations** and the tracking of internal **program variable values** into consideration.
- **Lattice-Based Reachability:** Evaluates global control invariants across abstract domain structures.

Assertion Failure Interpretation at State 3

Case A: Diverging Process Automata





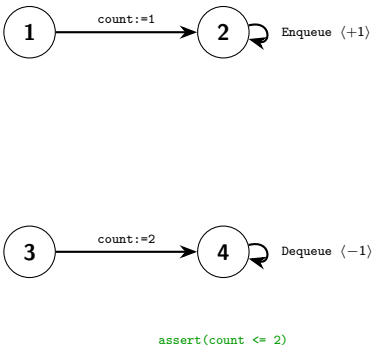

```
assert(count <= 2)
```

Lattice Invariant Tracking Profile

- **Assertion Point:** State 3
- **Product States:** $(\mathbf{1}, \mathbf{3})$ and $(\mathbf{2}, \mathbf{3})$
- **Value of 'count' in these states:** Includes $\{[0, 0], [0, 1], [0, 2]\}$
- **Widening Impact:** Interleaved loop execution steps trigger widening to $[-\infty, +\infty]$, generating Assertion Failure.

Assertion Success Interpretation at State 4

Case B: Bounded Precise Automata



Precise Invariant Tracking Profile

- **Assertion Point:** State 4
- **Product States:** $(\mathbf{1}, \mathbf{4})$ and $(\mathbf{2}, \mathbf{4})$
- **Value of 'count' in these states:** Kept tightly inside $[\mathbf{0}, \mathbf{2}]$
- **Widening Impact:** The variable does not undergo premature widening here since they were not part of a looping structure, meaning the Assertion Passes.

Conclusion & Ongoing Work

In summary, effective concurrent verification requires balancing robust analysis precision with optimized execution efficiency.

Ongoing Work:

- **Preserve Analysis Precision:** Prevent false-positive assertion failures by identifying and controlling premature widening.
- **Optimize Efficiency Metrics:** Drastically reduce overall termination time through targeted state-space pruning.

Thank You